IT4010 Research Project – 2026 Jan

Topic Assessment Form

V2.1

Project ID:

1. Topic (12 words max)

Auto Forge: Multi-Agent LLM System to Automate Requirements-to-Prototype Software Development

2. Research group the project belongs to

SST - Software Systems & Technologies

3. Specialization of the project belongs to

Software Engineering (SE)

4. If a continuation of a previous project:

Project ID	
Year	

5. Brief description of the research problem including references (300 words max) – references not included in word count.

Modern software development requires coordinated work across roles (business analyst, architect, UI/UX, developer, QA, DevOps, security). However, teams frequently struggle with unclear requirements, weak traceability, and delayed verification, which often leads to rework, higher cost, and quality/security issues. Recent LLM-based "software agents" show strong potential to automate parts of this workflow—such as planning, design, coding, testing, and issue fixing—using role-based collaboration and tool use. For example, multi-agent "software company" style frameworks such as **ChatDev** and **MetaGPT** demonstrate how specialized agents can coordinate to produce software artifacts, but they can still be fragmented, highly prompt/tool dependent, and may not consistently enforce an end-to-end SDLC pipeline with strong governance and traceability from requirements to a validated prototype [1], [2].

Similarly, agent systems that operate inside real repositories (e.g., **SWE-agent**) and iterative testing-based multi-agent coding approaches (e.g., **AgentCoder**) show practical automation for development tasks, yet they are commonly evaluated around narrower engineering goals (issue resolution and code-generation loops) rather than a business-approved requirements-to-prototype flow with explicit approval gates and artifact linkage [4], [5]. Benchmarking studies like **SWE-bench** reinforce that resolving real GitHub issues demands repository-level understanding, multi-file edits, and dependable validation—highlighting why robust evaluation is essential and why end-to-end automation remains challenging [7]. In addition, work such as **CodeAct** suggests that allowing agents to take executable actions and revise behavior based on observed outcomes can improve reliability compared to brittle, one-shot generation—supporting the need for safe, tool-driven execution loops within governed agent pipelines [8]. More recent studies also show two important directions: **MASAI** explores a modular architecture with specialized sub-agents for software engineering tasks, aligning with the idea that structured decomposition can improve capability and controllability [9], while **Agentless** argues that strong performance can sometimes be achieved with simpler, well-scoped pipelines (e.g., localization → repair → validation), which is valuable when designing baselines and ensuring practicality [10]. Finally, generalist multi-agent software engineering frameworks (e.g., **HYPERAGENT**) further emphasize the need for controllable, reliable, and explainable agent pipelines to support real-world development governance and safe execution [6].

The future users of this research include:

(i) **Business Analysts / Product Owners** who need consistent, auditable SRS/SDS outputs,
(ii) (ii) **student teams / startup teams** with limited expert resources, and
(iii) (iii) **software organizations** aiming to reduce repetitive engineering workload while improving quality and governance.

The impact is **technical** (higher requirement clarity, better test pass rates, fewer regressions), **economic** (reduced engineering hours and rework), and **ethical/legal** (need for human approval gates, sandboxed tool execution, and secure-by-design checks when agents modify code—especially for security-sensitive changes) [4], [6]. The project aligns with **SDG 9 (Industry, Innovation and Infrastructure)** by improving

software productivity and reliability, and **SDG 8 (Decent Work and Economic Growth)** by augmenting developers with safer automation and reducing repetitive tasks.

References

[1] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, Maosong Sun "**ChatDev: Communicative Agents for Software Development",** pp.1-5, arXiv:2307.07924v5 [cs.SE] 5 Jun 2024

[2] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, Jürgen Schmidhuber, "**MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework",** arXiv:2308.00352v7 [cs.AI] 1 Nov 2024

[3] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, Graham Neubig, "**OpenHands: An Open Platform for AI Software Developers as Generalist Agents**", arXiv:2407.16741v3 [cs.SE] 18 Apr 2025

[4] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, Ofir Press ,"**SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering**," arXiv:2405.15793 [cs.SE], 11 Nov 2024

[5] Dong Huang, Jie M.Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, Heming Cui, **"AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation,"** arXiv:2312.13010v3 [cs.CL] 24 May 2024.

[6] Huy Nhat Phan, Tien N. Nguyen, Phong X. Nguyen, Nghi D. Q. Bui, **"HYPERAGENT: Generalist Software Engineering Agents to Solve Coding Tasks at Scale,"** arXiv:2409.16299 [cs.SE] 5 Sep 2025.

[7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, Karthik Narasimhan **"SWE-bench: Can Language Models Resolve Real-World GitHub Issues?"** arXiv:2310.06770, 2023.

[8] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, Heng Ji **"Executable Code Actions Elicit Better LLM Agents"** arXiv:2402.01030, 2024.

[9] Daman Arora˚, Atharv Sonwane˚, Nalin Wadhwa˚ Abhav Mehrotra, Saiteja Utpala, Ramakrishna Bairi Aditya Kanade, Nagarajan Natarajan, "**MASAI: Modular Architecture for Software-engineering AI Agents,"** arXiv:2406.11638v1 [cs.AI] 17 Jun 2024

[10] Chunqiu Steven Xia∗ Yinlin Deng∗ Soren Dunn Lingming Zhang, **"Agentless: Demystifying LLM-based Software Engineering Agents",** arXiv:2407.01489v2 [cs.SE] 29 Oct 2024

6. Brief description of Existing Research and Systems (300 words max)

Recent research shows multiple approaches to agentic software development. **ChatDev** coordinates specialized LLM agents through structured communication across design–coding–testing, using language as a shared interface to reduce handoff friction between phases. **MetaGPT** models a "software company" by encoding **SOP-like (Standard Operating Procedure) pipelines** and generating structured intermediate artifacts (e.g., specs, designs) to improve reliability and reduce coordination errors. **OpenHands** (formerly **OpenDevin**) emphasizes execution: agents act like developers by editing files, running commands, and browsing the web inside sandboxed environments, often paired with benchmark-based evaluation. **AgentCoder** demonstrates iterative multi-agent development (programmer + test designer + test executor) where automated test feedback continuously refines code quality.

In industry, agentic tools such as **Lovable** (rapid app/prototype generation), **Cursor** (agentic IDE that can understand a codebase, edit multiple files, and run tasks), and **Kiro** (spec-driven development using "specs/steering/hooks") significantly accelerate implementation. However, these tools are typically optimized for quickly shipping code or prototypes rather than enforcing end-to-end SDLC governance with mandatory **Business Analyst (BA) approval gates**, domain-first requirement discovery, and complete artifact traceability.

Gaps / limitations identified:

1. Many systems focus on **issue fixing** or partial SDLC stages, not a **requirements-to-prototype** pipeline with explicit BA approval gates.
2. Limited support for **domain knowledge acquisition** as a first-class module (business workflows, constraints) before design/coding.
3. Weak **traceability** across artifacts (SRS → SDS → architecture → code → tests → security findings).
4. Security is often handled as an afterthought; stronger integration of **architectural threat checks + code scanning** is needed.

7. Brief description of the solution's nature, including a conceptual diagram (maximum 500 words).

Proposed solution: AutoForge — a human-in-the-loop, multi-agent LLM framework that automates a structured SDLC pipeline from business requirements to a validated prototype. AutoForge addresses gaps by (i) enforcing approval gates, (ii) incorporating domain knowledge agents, (iii) producing versioned artifacts with traceability, and (iv) running verification + security checks before delivery.

System Perspective (Input → Process → Output)

Inputs

- BA interview script / feature request / constraints
- Domain documents (e.g., fintech workflow notes)
- Target stack selection (e.g., Node/Express + React)

Process

1. Requirements Specification Bot chats with BA → generates SRS (FR/NFR/acceptance criteria) → BA approves
2. Domain Knowledge Agents build workflow/rule pack → feed into SDS
3. Auto agentic pipeline generates:
    - SDS (detailed dev spec) → approval
    - Architecture pack (C4/ADR/API contracts)
    - UI/UX pack (user flows, wireframes)
    - DB pack (ERD/schema)
    - Code prototype + tests
    - DevOps pack (CI/Docker)
    - Security pack (threat scan + SAST/dependency scan)
4. Prototype + reports returned to BA for review.

Outputs

- Approved SRS + SDS
- Working prototype (limited scope features)
- Tests + CI pipeline
- Security findings + mitigations
- Traceability matrix (FR → design/code/test/security)

Diagram 1 (workflow + agents):

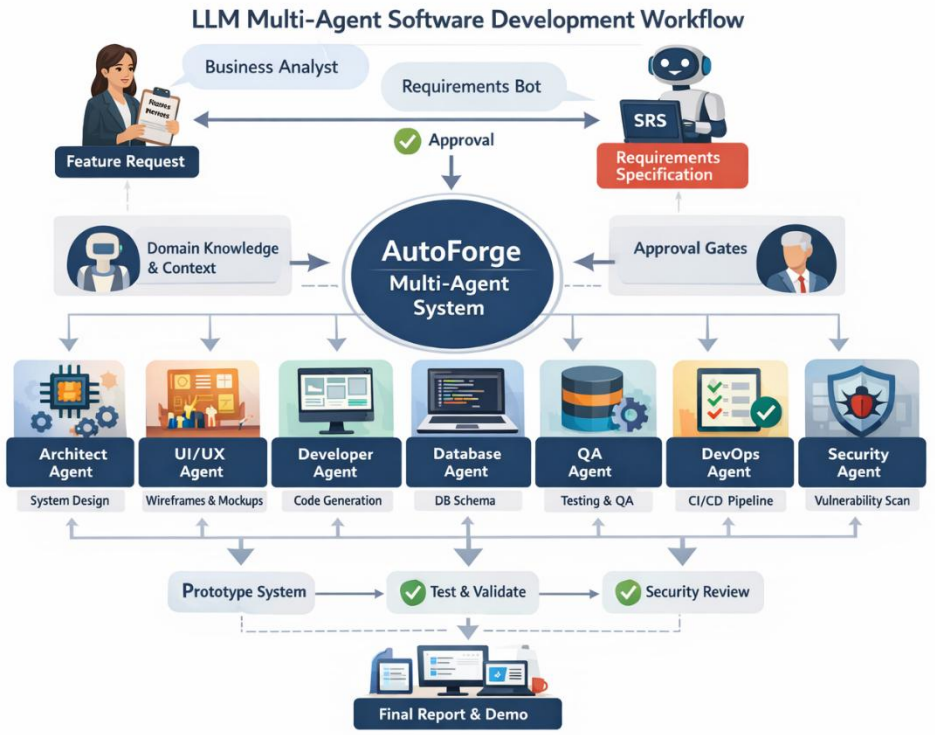
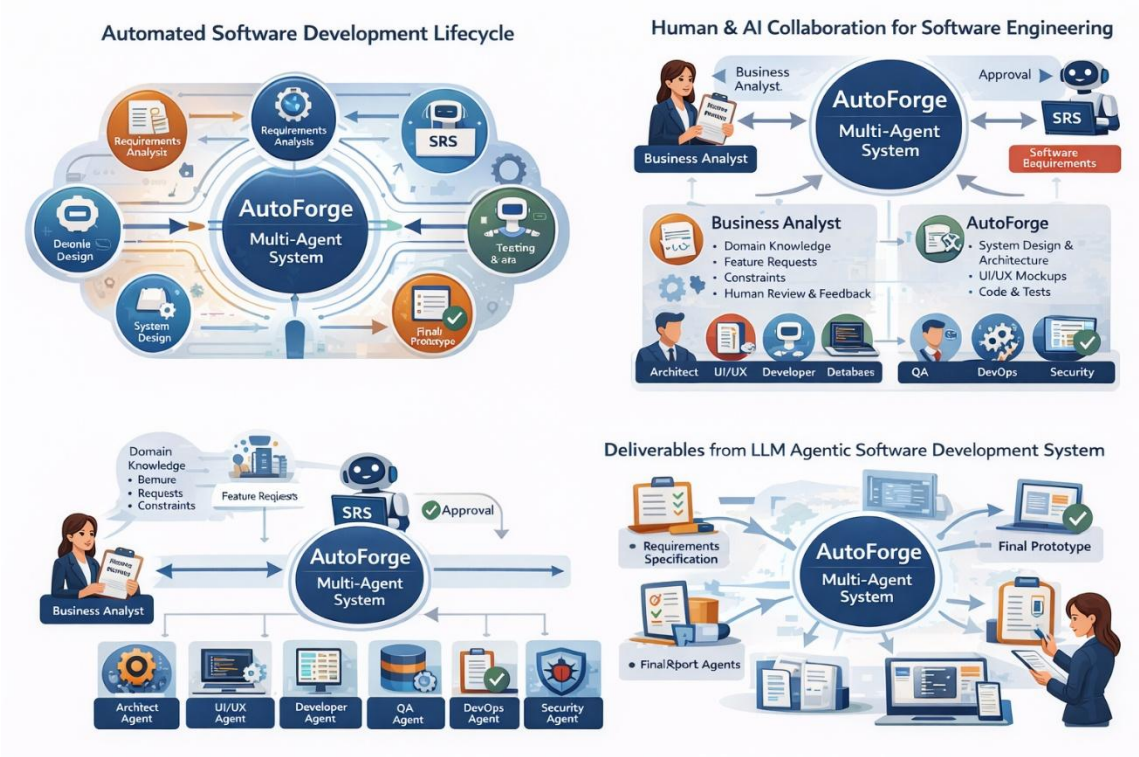


Diagram set (multiple variants):



8. Brief overview of the availability of necessary specialized domain expertise, knowledge, and data. (500 words max)

The project requires "software process + requirements engineering + agent workflows." Team members can build domain knowledge via (i) short expert interviews (a BA/PM mentor), (ii) curated domain reading packs (fintech/marketing workflows), and (iii) iterative validation of SRS/SDS outputs through supervisor feedback.

Technical knowledge required:

- LLM agent orchestration (state machine / multi-agent roles)
- Retrieval (RAG) for repo + domain docs
- Tool execution for tests/lint/build in sandbox
- Security scanning and reporting
  This can be gained via official documentation and small prototypes (e.g., agent that writes + runs tests) before integrating full pipeline.

Data collection:

- Public research datasets/benchmarks for issue-fixing and evaluation (e.g., SWE-bench)
- Public code repositories (open-source) for controlled experiments
- Synthetic requirement tickets + BA-crafted interview scripts for requirement evaluation
- Logs from AutoForge runs: iterations, tool outputs, test results, security findings (used for metrics)

Ethical clearance:
If the project uses only public repositories and synthetic requirements, ethical risk is low. If you conduct interviews with real BAs or collect user feedback, obtain informed consent, avoid collecting personal/sensitive data, and anonymize responses. Human approval gates reduce risks from autonomous code changes.

9. Objectives and Novelty

Main Objective: Design and evaluate AutoForge, a human-in-the-loop multi-agent LLM system that converts business requirements into a validated software prototype with traceability, testing, and security checks.			
Member Name with Registration No	Sub Objective	Tasks	Novelty
Santhuka D.N.M.D, IT22098078	Requirements Bot + Domain Agents + Orchestrator	• Build Requirements Bot to generate **SRS** + acceptance criteria • Implement Domain Knowledge ingestion (workflow/rules pack) • **Output**: SRS + domain pack + traceability tags (FR IDs) • Build orchestrator workflow (states, approvals, artifact versioning)	Domain workflow agents(Pull info from IEE) + BA approval gate integrated before development (reduces hallucination and requirement drift). Enforced SDLC pipeline with structured artifacts (SRS→SDS→design) rather than ad-hoc multi-agent chat.

Sansala T.G.B.D, IT22174826	Architect/UIUX Agents	• Implement Architect agent (API contract) • Implement UI/UX agent (flows + wireframes +)	AutoForge will generate a complete Design Pack from the approved SRS, including an OpenAPI/Swagger API contract, UML diagrams (use case, class, sequence, object), and UI/UX outputs (user flows and wireframes). This pack will be exported as a structured deliverable to support review and to help downstream development start with clear, consistent design documentation.
Dissanayake T.C, IT22157232	Development Agent(Dev/DB /DevOps Agents + Tool Runner)	• Implement Developer agent to generate prototype code (scoped features) • DB agent • DevOps agent (Docker + CI)	Patch-Only Code Editing (No Full File Rewrite) Novelty: Developer agent is only allowed to output diffs/patches, not full rewritten files. Why it's special: Makes changes reviewable, reduces accidental loss of code, enables rollback and PR-style review.
Oberathna R.D.T.D, IT22102478	Security Agent + Evaluation Framework + + "Compliance & Traceability Dashboard" + Testing Agent	• Threat modelling checks + static analysis integration • Dependency scanning and security report generation	Security integrated into both architecture + code stages; evaluation aligned with reliable benchmarks and human validation style.

10. <u>This part is to be filled by the Supervisor and the Co-supervisor of the Project.</u>

a) This research topic possesses a comprehensive scope suitable for a final-year project.

Yes		No	

b) The proposed topic exhibits novelty.

Yes		No	

c) The student group can successfully execute the proposed project.

Yes		No	

d) The proposed sub-objectives reflect the students' areas of specialization.

Yes		No	

e) Any other comments:

	Title	First Name	Last Name	Signature
Supervisor				
Co-Supervisor				
External Supervisor				
Summary of external supervisor's (if any) experience and expertise				